

ML and DL

30-12-2025

Lab - 6

Decision Tree Classifier

```
In [ ]: from sklearn.datasets import load_iris
```

Import iris data set using sklearn

```
In [ ]: data = load_iris()
```

```
In [ ]: data
```

```
In [ ]: type(data)
```

Importing the libraries

```
In [ ]: import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sb
```

Divide the data into input and output

```
In [ ]: data.target
```

```
In [ ]: data.feature_names
```

```
In [ ]: data.target_names
```

```
In [ ]: x = data['data']
```

```
In [ ]: y = data['target']
```

Splitting the dataset into the Training set and Test set

```
In [ ]: from sklearn.model_selection import train_test_split
```

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x,y,random_state=42,test_size=0
```

```
In [ ]: y
```

Fitting DecisionTreeClassifier on dataset

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
```

```
In [ ]: model = DecisionTreeClassifier(splitter='random')
```

```
In [ ]: model.fit(x_train,y_train)
```

Display Decision Tree

```
In [ ]: from sklearn import tree
```

```
In [ ]: %matplotlib inline
```

```
In [ ]: tree.plot_tree(model)
```

whos variable memory ma che e jova

reset clear karva

```
In [ ]: %whos
```

Predict the x_test

```
In [ ]: model.predict(x_test)
```

```
In [ ]: y_test
```

```
In [ ]: model.score(x_train,y_train)
```

```
In [ ]: model.score(x_test,y_test)
```

Import diabetes.csv dataset

```
In [ ]: df = pd.read_csv('diabetes.csv')
```

```
In [ ]: df
```

Check the distribution of the target

```
In [ ]: df['Outcome'].value_counts()
```

```
In [ ]: df.head()
```

Replace zeros with NaN for specific columns

```
In [ ]: column_to_replace = ['Glucose', 'BloodPressure', 'SkinThickness', 'BMI', 'Insulin']  
df[column_to_replace] = df[column_to_replace].replace(0,np.nan)
```

```
In [ ]: df
```

Check for missing values

```
In [ ]: df.isnull().sum()
```

Fill missing values with median

```
In [ ]: cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
```

```
In [ ]: for col in cols:  
    df[col] = df[col].fillna(df[col].median())
```

```
In [ ]: df.isnull().sum()
```

Visualize Distributions

```
In [ ]: sb.pairplot(df,hue='Outcome')
```

Convert Target data into interger code

```
In [ ]: df['Outcome'] = df['Outcome'].map({'Diabetic':1,'Non Diabetic' :0})
```

```
In [ ]: df
```

```
In [ ]: sb.heatmap(df.corr(),annot=True,cmap='coolwarm')  
plt.show()
```

Divide the data into input and output

```
In [ ]: x = df.drop('Outcome',axis=1)
```

```
In [ ]: x
```

```
In [ ]: y = df['Outcome']
```

Splitting the dataset into the Training set and Test set

```
In [ ]: x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.3,random_state=42)
```

Create Model

```
In [ ]: model_tree = DecisionTreeClassifier()
```

Fitting DecisionTreeClassifier on dataset

```
In [ ]: model_tree.fit(x_train,y_train)
```

Display Decision Tree

```
In [ ]: from sklearn import tree
```

```
In [ ]: %matplotlib inline
```

```
In [ ]: tree.plot_tree(model_tree)
```

Predict the x_test

```
In [ ]: y_predict = model_tree.predict(x_test)
```

Display Training Accuracy

```
In [ ]: model_tree.score(x_train, y_train)
```

Display Test Accuracy

```
In [ ]: model_tree.score(x_test, y_test)
```

Confusion Matrix

```
In [ ]: from sklearn.metrics import confusion_matrix
```

```
In [ ]: cm = confusion_matrix(y_test, y_predict)
```

```
In [ ]: cm
```

Save Model State

model ne disk na store karvu

```
In [ ]: import pickle
```

```
In [ ]: with open('model.pkl','wb') as file:  
        pickle.dump(model,file)
```

Load Model from Disk

```
In [ ]: with open('model.pkl','rb') as file:  
        loaded_model = pickle.load(file)
```

```
In [ ]: loaded_model.predict(x_test)
```